EVENT-TICKETING

Complete system documentation

Author: Hazem-Musallam · **Stack:** Spring Boot 3.4 · MySQL 8 · Redis · Flutter · Thymeleaf

Scope: HLD and LLD — architecture, database design, domain entities, DTOs, services, controllers, scheduled jobs, caching, rate-limiting, authentication, business flows and known gaps.

Reading approach: each chapter stands alone. Start with Chapter 1 (business context) and Chapter 2 (architecture) if new to the system; jump to any later chapter as a reference.

Generated with simple language and plain-English explanations for every file, class, and flow.

Table of contents

Chapter 1 What the system does (business context)

Chapter 2 High-Level Design (architecture)

Chapter 3 Technology stack

Chapter 4 Database design — schema and ERD

Chapter 5 Domain entities (every @Entity explained)

Chapter 6 DTOs — request and response shapes

Chapter 7 Repositories (JPA data access)

Chapter 8 Services — business logic layer

Chapter 9 Business flows (sequence diagrams)

Chapter 10 Controllers — REST API reference

Chapter 11 Cross-cutting concerns

Chapter 12 Configuration reference

Chapter 13 Known gaps and risks

Chapter A Glossary

Chapter B API examples

Chapter C Concurrency deep-dive

Chapter D FAQ

Chapter 1 — What the system does

Event-Ticketing is a backend for selling tickets to live events. Think of a cinema or concert booking website: the user picks an event, picks a seat (or a quantity for general-admission), pays, and gets a ticket.

The actors

Actor	What they do
Mobile user (customer)	Browses events, holds seats, pays, gets a ticket with a QR code.
Admin (organizer)	Creates halls, places sections, defines events, sets prices per section.
Scheduled jobs	Release abandoned holds; reconcile uncertain payments.
Payment gateway	Processes the actual card charge (fake gateway in dev, real provider in prod).

The core problem it solves

Selling the same seat twice is a disaster: the second buyer paid for nothing, and you owe them a refund. The system's main job is to make that physically impossible, even when 500 users click the same seat in the same millisecond. A second job is to not let one seat sit 'held' forever if the user walks away mid-checkout — someone else should get a chance to buy it.

The two invariants:
1. No double-booking. At most one booking may hold any given seat at any moment.
2. No stuck inventory. An unpaid hold must release automatically within 10 minutes.

Why both are hard

Double-booking is hard because concurrent requests race. Two users click seat A1 at the exact same moment; without protection, both checks say 'free', both inserts succeed, and the database now has two holds on A1. The system prevents this with a **unique database index on an auto-generated column** (explained in Chapter 4) — the database itself refuses to insert the second row.

Stuck inventory is hard because the user controls when to pay, not the server. If they close the browser, nothing tells us. The system solves this with a **scheduled sweeper job** that every 30 seconds releases any hold whose expires_at is in the past.

What makes this design 'first-class'

Many ticketing projects start simple and get painted into corners. This one anticipated several:

- **Sections as first-class entities**: not a label slapped on seats. A section has its own name, booking mode (seated or general admission), capacity, currency, and polygon shape in the hall.
- **Mixed booking modes per event**: one hall can have VIP seated rows AND a general-admission standing area. One booking can hold both at once.
- **Prices live on EventPricing, not sections**: the same section (VIP) can cost 50 JOD for one event and 80 JOD for another. The *hall* describes the venue; the *event* overlays the price.
- **Snapshotting at hold time**: price and section name are copied onto the booking row. A later price edit or section rename never corrupts a live booking.

- **Payment attempts are first-class**: every retry is its own row with its own idempotency key, so a retried charge can never double-bill.

Chapter 2 — High-Level Design

The system is a Spring Boot monolith with three logical modules (**catalog**, **reservation**, **payment**) and a thin **web** layer. State lives in MySQL; Redis is used for caching read models and counting hold attempts. Two scheduled jobs quietly keep the system healthy.

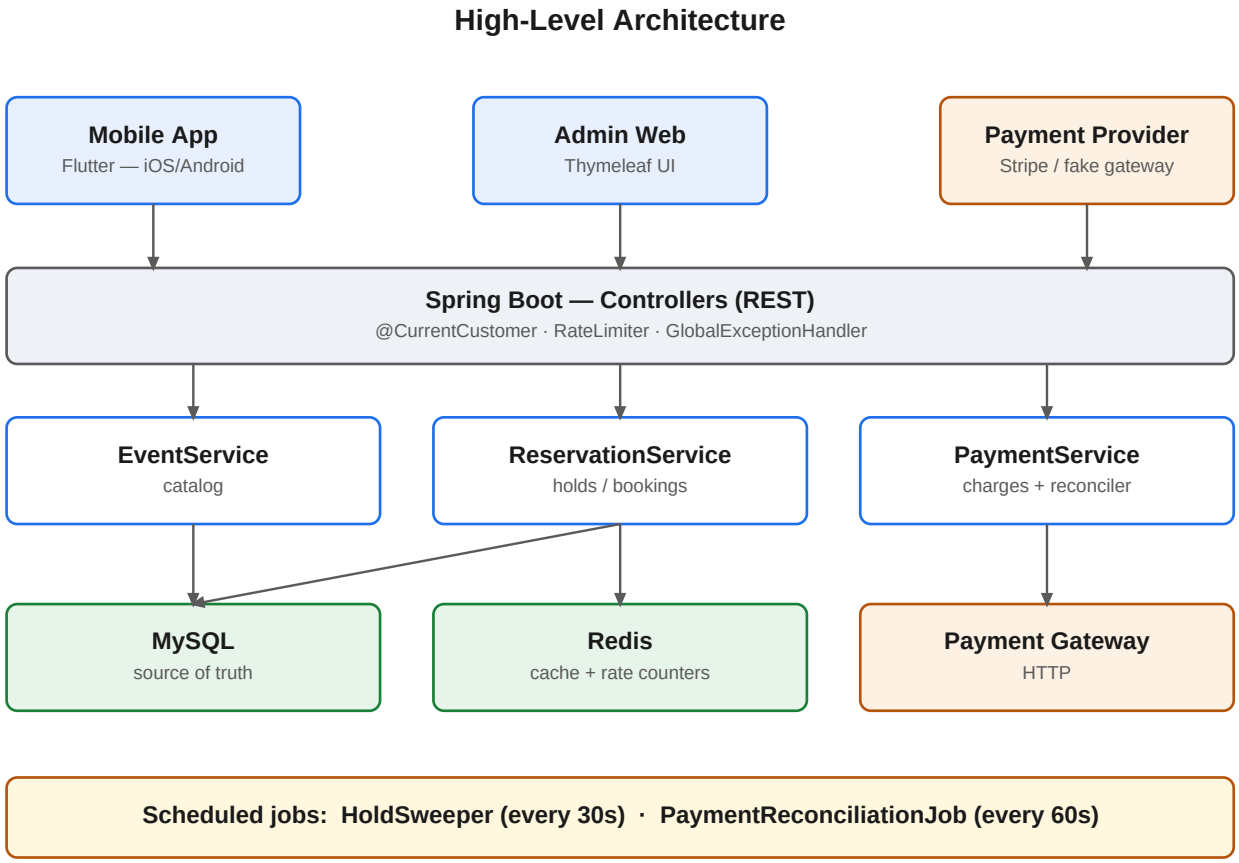


Figure 2.1 — Logical architecture.

The three modules

Module	Owns	Main services
catalog	The venue setup: organizers, halls, sections, seats, events, per-event pricing.	EventService, HallService, LayoutPresetService, OrganizerS
reservation	Orders: holding seats, confirming them, cancelling, issuing tickets.	ReservationService, HoldSweeper
payment	Charge attempts, reconciliation, refunds.	PaymentService, PaymentGateway, PaymentReconciliation

Dependency direction

Reservation depends on Catalog (it needs to know what an event and a seat are). Payment depends on Reservation (it confirms a booking after a successful charge). Catalog depends on nothing — it is the foundation. Nothing in catalog knows about bookings or payments.

Request paths

Two surfaces talk to the backend:

- **Mobile app** (/api/**): browse events, view seat map, create booking, pay, view tickets. Controllers live in web.mobile.
- **Admin web** (/admin/**): server-rendered Thymeleaf pages. Controllers live in web.admin. Separate codebase at ../admin/; this backend exposes its REST endpoints.

What the diagram leaves out

File uploads for event posters are served from the local filesystem by UploadController — fine for development, but in production that directory needs to be a mounted volume or object storage. The WebConfig allows CORS from any origin with no credentials, which is the default for a mobile app talking to a public API but would need tightening if browser-based admin ever talks to the same endpoints with cookies.

Chapter 3 — Technology stack

Layer	Tech	Why
Language	Java 21	Records, pattern matching, modern switch.
Framework	Spring Boot 3.4.1	Mature, boring, hiring-pool friendly.
Persistence	Spring Data JPA + Hibernate	Entity-centric; integrates with transactions.
Migrations	Flyway	Schema as code. <code>ddl-auto: validate</code> forbids drift.
Database	MySQL 8.x	Generated columns, STORED unique indexes with NULL.
Cache / counters	Redis 7	Low-latency reads; INCR for rate limiter.
API docs	springdoc-openapi	OpenAPI 3 spec auto-generated at <code>/v3/api-docs</code>.
Mobile	Flutter	Separate repo; consumes <code>/api/**</code>.
Admin UI	Thymeleaf (separate repo)	Server-rendered HTML; no SPA bundle.
Payment	Fake gateway (dev) + interface	Swap in Stripe / Checkout.com behind the <code>PaymentGateway</code>

Why no Spring Security

A full Spring Security setup was traded for a lightweight @CurrentCustomer resolver: a parameter annotation on controller methods that produces a CustomerId from the request. Simpler to wire, simpler to test. The trade-off: no CSRF, no OAuth filter chain, no method-security annotations. Good enough for mobile-first (where auth is token-based and CSRF is not relevant) but a limitation to note.

Current state: the resolver class (CurrentCustomerArgumentResolver), the token authenticator, the dev-token controller, and the WebMvcConfig that registered the resolver are all **deleted** in the working tree. Every endpoint using @CurrentCustomer would 500 at runtime until the resolver is replaced. This is a known gap tracked in Chapter 13.

Chapter 4 — Database design (expanded)

The schema evolved over 16 Flyway migrations. This chapter walks through them in order, explains what each one solved, and includes a dedicated deep-dive into the active_lock concurrency mechanism — the single cleverest piece of SQL in the system.

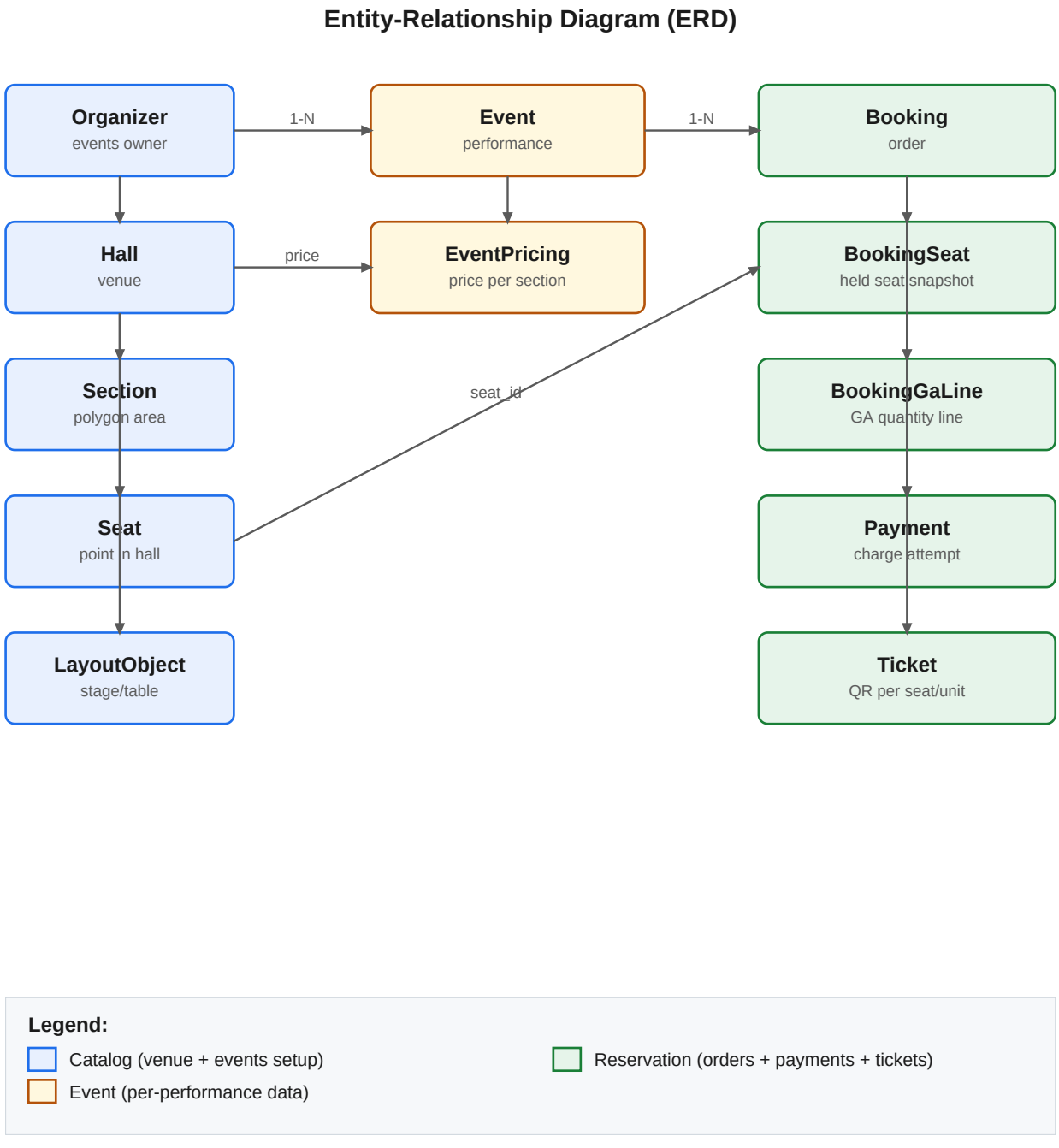


Figure 4.1 — ERD across the three logical clusters.

The migration ladder

Every V*.sql file is a schema change applied exactly once per database, in filename order. Flyway refuses to start if any applied migration's checksum differs from the file on disk — so migrations are append-only, never edited after they ship.

Migration	Title	What changed and why
V1	Catalog schema	Creates organizer, hall, seat, event, event_pricing. Early price model: per-seat-type. Seats are
V2	Reservation schema	Creates booking and booking_seat. Introduces the active_lock generated column and
V3	Payment	Creates the payment table. One row per booking (UNIQUE booking_id) and one row per idem
V4	Seat layout columns	Adds layout_x, layout_y, rotation, width, height to seat for the 3D hall editor. Capacity-neutral -
V5	layout_object table	Non-bookable décor: stages, tables, walls, bars. Stored in a separate table so they can never
V6	Sections and tickets	The big refactor: first-class <code>section</code> entities with name, booking_mode (SEATE
V7	layout_preset	Save-and-reuse full hall layouts. Payload stored as JSON, hall-agnostic (relative coords to the
V8	Remove seat types	Migrates pricing from (event, seat_type) to (event, section). Deletes legacy pricing that can't be
V9	Section price moves to event	Prices now live only on event_pricing — section.default_price is removed. The hall describes t
V10	Hot-path indexes	Composite indexes that match the hot-path queries exactly: (event_id, status, expires_at), (sec
V11	Customer hold index	Adds idx_booking_customer_status_expiry (customer_ref, status, expires_at) — serves the si
V12	Attempt-scoped idempotency keys	A payment provider treats an idempotency key as 'replay the stored response'. With a key of 'b
V13	Payment dead-letter columns	Adds reconcile_attempts, last_reconcile_at, last_reconcile_error to payment. Without this, a re
V14	GA fields on booking	First attempt at mixed booking modes: adds general_admission_quantity, general_admission_
V15	booking_ga_line table	A booking may need to hold GA tickets in several sections at once. Moves the single-section G
V16	Event media and tags	Adds image_url, tagline, featured on event. Creates the event_tag table (ElementCollection). E

The active_lock mechanism, step by step

This is the single most load-bearing piece of SQL in the system. It is the reason two users cannot double-book the same seat, no matter how closely their inserts race.

The declaration

```
active_lock VARCHAR(48) GENERATED ALWAYS AS (
    CASE WHEN status IN ('HELD', 'BOOKED')
         THEN CONCAT(event_id, '-', seat_id)
         ELSE NULL END
) STORED,
CONSTRAINT uq_booking_seat_active UNIQUE (active_lock)
```

Line-by-line

Line	Meaning
<code>GENERATED ALWAYS AS (...)</code>	MySQL computes this column's value from the expression on every row insert or update. The
<code>STORED</code>	Writes the computed value to disk. Virtual (not stored) columns are recomputed on every read
<code>CASE WHEN status IN ('HELD', 'BOOKED')</code>	The 'currently live'. Only seats currently reserved should block other reservations. R
<code>THEN CONCAT(event_id, '-', seat_id)</code>	The key. Two different events at the same seat are independent — '42-2001' and '43-2
<code>ELSE NULL</code>	The escape hatch. MySQL's UNIQUE index allows unlimited NULLs. When a row goe
<code>UNIQUE (active_lock)</code>	The index. Enforces that no two rows can have the same non-NULL active_lock value. Enforc

What happens during a race

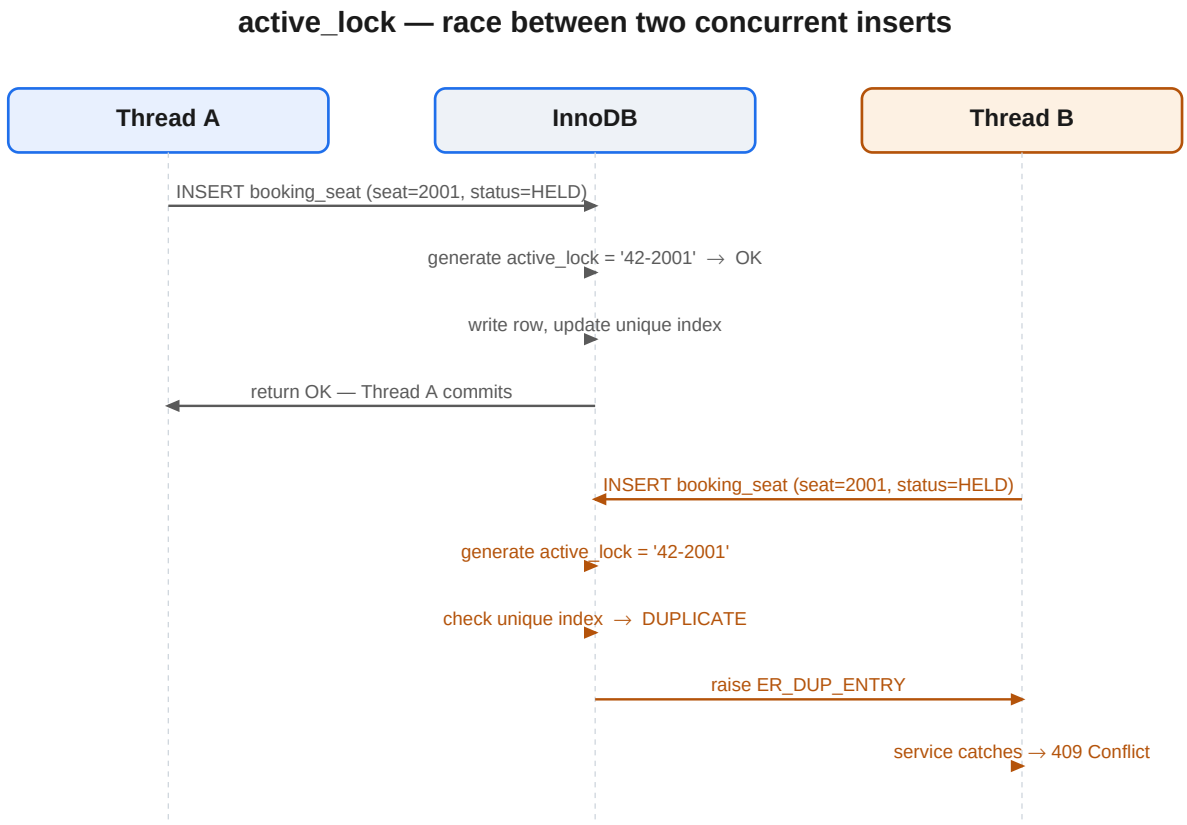


Figure 4.2 — Two threads attempt to hold seat 2001 for event 42.

The sequence above assumes Thread A commits a nanosecond before Thread B tries its insert. What if they're truly simultaneous?

InnoDB serializes unique-index inserts at the row-lock level inside the index B-tree. If both threads try to insert the same active_lock value in overlapping transactions, the first one gets the index entry and the second waits. Once the first commits, the second wakes up, sees the conflict, and gets ER_DUP_ENTRY. If the first rolls back, the second wakes up and succeeds. Either way, exactly one wins. Zero coordination from the application.

Why not a trigger?

You could implement the same logic with a BEFORE INSERT trigger that runs SELECT COUNT(*) FROM booking_seat WHERE event_id=? AND seat_id=? AND status IN ('HELD','BOOKED') and raises an error if >0. Three reasons not to:

- **Race condition inside the trigger**: the SELECT and INSERT are separate statements. Two concurrent inserts can both SELECT zero before either inserts. You'd need SERIALIZABLE isolation, which murders throughput.
- **Not declarative**: a developer reading the schema sees a booking_seat table and no obvious guard. The invariant lives in trigger code elsewhere, easy to miss or break in a refactor.
- **Not portable in principle**: a generated column + unique index is standard SQL (2003); triggers have database-specific syntax.

Why not a PostgreSQL partial unique index?

Postgres lets you write:

```
CREATE UNIQUE INDEX uq_booking_seat_active
ON booking_seat (event_id, seat_id)
WHERE status IN ('HELD', 'BOOKED');
```

Cleaner syntax, same semantics. MySQL doesn't support partial indexes, so the generated-column trick is the equivalent. If this system ever migrates to Postgres, V2 becomes two lines instead of six.

Indexes, mapped to the queries they serve

Index	Query it supports	Where
uq_booking_seat_active	Double-booking guard (the write path)	V2
idx_booking_status_expires	<code>HoldSweeper.findExpiredHeldBatch</code>: status='PENDING_PAYMENT'	V2
idx_booking_seat_event_status	<code>getSeatMap</code>: which seats are HELD/BOOKED on event X	V2
idx_booking_event_status_expiry	<code>sumConfirmedQuantity / sumReservedQuantity</code>: capacity check unde	V10
idx_booking_customer_created	<code>findByCustomerRefOrderByCreatedAtDesc</code>: 'my bookings' list, DESC	V10
idx_payment_status_updated	<code>PaymentReconciliationJob.findPaymentsToReconcile</code>: status='INITIA	V10
idx_booking_customer_status_expiry	<code>countOpenHoldsByCustomer</code>: rate-limit check at booking creation	V11
idx_payment_reconcile_queue	<code>reconcile</code> queue with exponential backoff	V13
idx_booking_ga_line_event_section	Per-(event, section) GA capacity aggregates	V15
idx_event_featured	Home-screen 'Featured' query	V16

The invariant map

What each guarantee actually comes from:

Invariant	Where it is enforced
No two bookings hold the same seat at once	DB unique index on active_lock (V2)
GA section capacity is never exceeded	Event row lock (PESSIMISTIC_WRITE on event) + COUNT(*) inside the transa
One customer can't open 100 unpaid holds	UnpaidHoldThrottle + Redis counter (10 min window, limit 5)
Price doesn't change under the customer mid-checkout	Snapshot: price copied onto booking_seat / booking_ga_line at hold time
Retry a declined payment without replaying the decline	Attempt-scoped idempotency key: 'booking-{id}-{attempt}' (V12)
Payment stuck in INITIATED is resolved eventually	PaymentReconciliationJob + exponential backoff (V13)
Broken payment doesn't consume gateway calls forever	reconcile_attempts; after 6 failures → NEEDS_REVIEW (V13)
Two concurrent updates to the same booking don't overwrite each other	@Version column (optimistic lock)

Chapter 5 — Domain entities (expanded)

This chapter shows the actual Java code of every @Entity class, with per-field explanations and the JPA annotations that make each mapping work. Every entity extends BaseEntity, which supplies the four columns every row has: id, version, created_at, updated_at.

BaseEntity — the common superclass

src/main/java/com/eventticketing/common/domain/BaseEntity.java

```
@Getter
@Setter
@MappedSuperclass
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Version
    @Column(name = "version", nullable = false)
    private Long version;

    @CreationTimestamp
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    @UpdateTimestamp
    @Column(name = "updated_at", nullable = false)
    private Instant updatedAt;
}
```

Annotation	Effect
<code>@MappedSuperclass</code>	This class is not a table itself. Its fields are inherited by every subclass's table.
<code>@Id + @GeneratedValue(IDENTITY)</code>	Primary key, auto-increment. MySQL fills in the value on insert.
<code>@Version</code>	Optimistic lock. Hibernate adds <code>WHERE version = ?</code> to every UPDATE and in
<code>@CreationTimestamp</code>	Hibernate sets this once, on insert. <code>updatable = false</code> prevents accidental over
<code>@UpdateTimestamp</code>	Hibernate rewrites this on every update.
<code>@Getter / @Setter</code>	Lombok — generates all getters and setters at compile time. No boilerplate in the source.

Hall — the venue

src/main/java/com/eventticketing/catalog/domain/Hall.java

A hall owns three child collections: seats, layout objects, and sections. All three are cascaded ALL + orphanRemoval: deleting the hall deletes everything, and removing an item from the collection deletes the row.

```
@Entity
@Table(name = "hall")
@Getter
@Setter
@NoArgsConstructor
public class Hall extends BaseEntity {

    @Column(name = "name", nullable = false)
```

```
    private String name;

    @Column(name = "address")
    private String address;

    @Column(name = "capacity", nullable = false)
    private int capacity;

    @Column(name = "is_seated", nullable = false)
    private boolean seated;

    @OneToMany(mappedBy = "hall", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Seat> seats = new ArrayList<>();

    @OneToMany(mappedBy = "hall", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<LayoutObject> layoutObjects = new ArrayList<>();

    @OneToMany(mappedBy = "hall", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Section> sections = new ArrayList<>();

    public void addSeat(Seat seat) {
        seat.setHall(this);
        this.seats.add(seat);
    }
    // ... addLayoutObject, addSection similarly
}
```

Decision	Why
<code>cascade = ALL</code>	Deleting a hall deletes its seats, sections, and layout objects. Updating a hall persists new chi
<code>orphanRemoval = true</code>	Removing a seat from <code>hall.seats</code> issues a <code>DELETE</code>. Without th
<code>mappedBy = "hall"</code>	The child owns the FK. Hall is the inverse side.
<code>addSeat</code> helper	Keeps the bidirectional reference consistent. Forgetting to set <code>seat.hall</code> would

Seat — a sellable spot

src/main/java/com/eventticketing/catalog/domain/Seat.java

```
@Entity
@Table(name = "seat", uniqueConstraints = {
    @UniqueConstraint(name = "uq_seat_hall_label", columnNames = {"hall_id", "label"})
})
public class Seat extends BaseEntity {

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "hall_id", nullable = false)
    private Hall hall;

    @Column(name = "row_label", nullable = false, length = 8)
    private String rowLabel;

    @Column(name = "row_index", nullable = false) private int rowIndex;
    @Column(name = "seat_number", nullable = false) private int seatNumber;
    @Column(name = "label", nullable = false, length = 16) private String label;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "section_id")
    private Section section;  // nullable — draft layouts

    @Column(name = "layout_x") private Integer layoutX;
```

```
    @Column(name = "layout_y") private Integer layoutY;
    @Column(name = "rotation_degrees") private Integer rotationDegrees;
    @Column(name = "layout_width") private Integer layoutWidth;
    @Column(name = "layout_height") private Integer layoutHeight;
    @Column(name = "section_name", length = 80)
    private String sectionName;  // denormalized snapshot
}
```

Key design notes: the FK to hall is **required** (optional = false, nullable = false) so a seat cannot exist orphaned. The FK to section is **nullable** because during hall creation the admin generates a grid of seats before drawing sections. Fetch is LAZY — loading a seat does not fetch its entire hall; Hibernate returns a proxy that fires a query on first access.

Section — the first-class area

src/main/java/com/eventticketing/catalog/domain/Section.java

Section carries no price (Chapter 4 explains why). Its distinguishing fields:

Field	Type	Notes
name	VARCHAR(120)	Free text. 'Royal', 'Balcony Left'. Not an enum.
bookingMode	ENUM(24)	SEATED or GENERAL_ADMISSION — stored as STRING.
currency	VARCHAR(8)	'JOD', 'USD', 'AED' — a venue property.
capacity	INT nullable	Required for GA; derived from seats for SEATED.
shapeKind	ENUM(16)	RECT, CIRCLE, POLYGON — a hint for the editor.
points	TEXT	JSON polygon: <code>[{"x":120,"y":40}, ...]</code> in hall-pixel space.
color	VARCHAR(16)	Hex, e.g. '#2E75DC'.

Event — a performance

src/main/java/com/eventticketing/catalog/domain/Event.java

```
@Entity
@Table(name = "event")
public class Event extends BaseEntity {
    @Column(name = "name", nullable = false) private String name;
    @Column(name = "description", length = 2000) private String description;
    @Column(name = "category", length = 80) private String category;
    @Column(name = "image_url", length = 500) private String imageUrl;
    @Column(name = "tagline", length = 200) private String tagline;
    @Column(name = "featured", nullable = false) private boolean featured = false;

    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "event_tag", joinColumns = @JoinColumn(name = "event_id"))
    @Column(name = "tag", length = 60, nullable = false)
    private Set<String> tags = new LinkedHashSet<>();

    @Column(name = "start_at", nullable = false) private Instant startAt;
    @Column(name = "end_at", nullable = false) private Instant endAt;

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false, length = 20)
    private EventStatus status = EventStatus.DRAFT;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "organizer_id", nullable = false)
```

```
    private Organizer organizer;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "hall_id", nullable = false)
    private Hall hall;

    @Column(name = "max_capacity", nullable = false)
    private int maxCapacity;

    @OneToMany(mappedBy = "event", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<EventPricing> pricing = new ArrayList<>();
}
```

EventStatus transitions: DRAFT → PUBLISHED → (SOLD_OUT | CANCELLED). maxCapacity is a snapshot of the hall's capacity at the time of event creation — it will not change if the hall is later edited (the event remains correct even after venue changes).

Booking — the order

src/main/java/com/eventticketing/reservation/domain/Booking.java

Status transitions: PENDING_PAYMENT → (CONFIRMED | EXPIRED | CANCELLED). Carries three child collections — seated lines, GA lines, issued tickets — each cascaded ALL + orphanRemoval.

```
@Entity
@Table(name = "booking")
public class Booking extends BaseEntity {
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "event_id", nullable = false)
    private Event event;

    @Column(name = "customer_ref", nullable = false)
    private String customerRef;

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false, length = 20)
    private BookingStatus status;

    @Column(name = "quantity", nullable = false) private int quantity;
    @Column(name = "total_amount", nullable = false, precision = 12, scale = 2)
    private BigDecimal totalAmount;

    @Column(name = "expires_at", nullable = false) private Instant expiresAt;
    @Column(name = "confirmed_at") private Instant confirmedAt;
    @Column(name = "payment_ref", length = 64) private String paymentRef;

    @OneToMany(mappedBy = "booking", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<BookingSeat> seats = new ArrayList<>();

    @OneToMany(mappedBy = "booking", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<BookingGaLine> gaLines = new ArrayList<>();

    @OneToMany(mappedBy = "booking", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Ticket> tickets = new ArrayList<>();

    public void addBookingSeat(BookingSeat seat) { seat.setBooking(this); seats.add(seat); }
    public void addGaLine(BookingGaLine line)    { line.setBooking(this); gaLines.add(line);
}

    public List<BookingSeat> heldSeats() {
        return seats.stream().filter(s -> s.getStatus() == BookingSeatStatus.HELD).toList();
    }
```

```
    public boolean isExpired(Instant now) {
        return expiresAt != null && !now.isBefore(expiresAt);
    }
}
```

BookingSeat — the held seat line

src/main/java/com/eventticketing/reservation/domain/BookingSeat.java

The entity that owns the active_lock column — but notice: active_lock is NOT mapped as a Java field. Hibernate's ddl-auto: validate would complain about an unmapped column if it weren't declared, so BookingSeat is explicitly annotated to tell Hibernate that these columns exist but belong to the database.

```
@Entity
@Table(name = "booking_seat")
public class BookingSeat extends BaseEntity {
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "booking_id", nullable = false)
    private Booking booking;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "event_id", nullable = false)
    private Event event;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "seat_id", nullable = false)
    private Seat seat;

    // snapshots at hold time
    @Column(name = "section_id") private Long sectionId;
    @Column(name = "section_name", length = 120) private String sectionName;
    @Column(name = "currency", length = 8) private String currency;
    @Column(name = "price", nullable = false, precision = 12, scale = 2)
    private BigDecimal price;

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false, length = 16)
    private BookingSeatStatus status;

    // active_lock is a generated STORED column in the schema — not mapped here.
    // It becomes event_id-seat_id while status is HELD or BOOKED, and NULL otherwise.
    // A UNIQUE index on active_lock makes double-booking a DB-level impossibility.
}
```

Payment — the charge attempt

src/main/java/com/eventticketing/reservation/domain/Payment.java

```
@Entity
@Table(name = "payment")
public class Payment extends BaseEntity {
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "booking_id", nullable = false)
    private Booking booking;

    @Column(name = "idempotency_key", nullable = false, length = 80, unique = true)
    private String idempotencyKey;  // 'booking-{id}-{attempt}'

    @Column(name = "customer_ref", nullable = false) private String customerRef;
    @Column(name = "amount", nullable = false, precision = 12, scale = 2)
    private BigDecimal amount;
```

```
    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false, length = 20)
    private PaymentStatus status;

    @Column(name = "attempt", nullable = false) private int attempt = 1;
    @Column(name = "reference", length = 64) private String reference;
    @Column(name = "failure_reason", length = 255) private String failureReason;

    // reconciliation state (V13)
    @Column(name = "reconcile_attempts", nullable = false) private int reconcileAttempts;
    @Column(name = "last_reconcile_at") private Instant lastReconcileAt;
    @Column(name = "last_reconcile_error", length = 500) private String lastReconcileError;
}
```

Enum summary

Enum	Values	Notes
EventStatus	DRAFT, PUBLISHED, SOLD_OUT, CANCELLED	Linear progression except for CANCELLED (terminal from
BookingStatus	PENDING_PAYMENT, CONFIRMED, EXPIRED, CANCELLED	PENDING_PAYMENT is the 'held' state. CONFIRMED is
BookingSeatStatus	HELD, BOOKED, RELEASED, CANCELLED	Only HELD and BOOKED contribute to the active_lock ge
PaymentStatus	INITIATED, SUCCEEDED, FAILED, REFUNDED	NEEDS_REVIEW is the dead-letter state after 6 f
SectionBookingMode	SEATED, GENERAL_ADMISSION	Decides whether a section has individual seats or just a c
SectionShape	RECT, CIRCLE, POLYGON	Hint for the admin editor — the actual boundary is always
LayoutObjectType	STAGE, TABLE, BAR, WALL, DECOR	Non-selling décor types.
SeatNumberingScheme	SEQUENTIAL, ODD_LEFT_EVEN_RIGHT, NUMBER_ONLY	Hint for the seat-numbering strategy.
SeatAvailabilityStatus	AVAILABLE, HELD, BOOKED	Projection used in the seat-map response — synthesized,

Every enum column is stored as VARCHAR with @Enumerated(EnumType.STRING), never ordinal. Rationale: reordering enum values in Java is a common refactor, and ordinal storage would silently remap every row in the database (status=2 now means something else). STRING is self-documenting and survives reordering.

Chapter 6 — DTOs

DTOs are plain data carriers at the API boundary. They separate the HTTP shape from the domain model, so the entity can evolve without breaking clients and clients don't see internal fields they shouldn't. Nearly all DTOs are Java records — immutable, concise, serialized by Jackson with zero boilerplate.

Categories

Kind	Convention	Examples
Request body	Named <code>Create*Request</code>	CreateBookingRequest, CreateEventRequest, ChangeSeatsReq
Response body	Named <code>*Response</code>	BookingResponse, EventResponse, EventSeatMapResponse
Nested / value items	Named <code>*Item</code>, <code>*Selection</code>	SeatLayoutItem, PricingItem, GeneralAdmissionSelection
Summary (list views)	Named <code>*SummaryResponse</code>	EventSummaryResponse, HallSummaryResponse

Reservation DTOs

CreateBookingRequest

What the mobile client sends to POST /api/bookings. Declares either a list of seat IDs (seated flow), a list of GA selections, or both. Has a hard cap MAX_SEATS = 10 enforced at the DTO level as defense in depth, so a huge JSON payload is rejected before any DB work.

```
public record CreateBookingRequest(
    @NotNull Long eventId,
    List<Long> seatIds,              // null or empty allowed if gaLines present
    List<GeneralAdmissionSelection> gaLines
) {
    public static final int MAX_SEATS = 10;
}
```

BookingResponse

What the server returns. Mirrors the Booking entity but with snapshotted price/currency per line, the expiresAt (so the UI can show a countdown), and the full seat and GA breakdown.

```
public record BookingResponse(
    Long id, Long eventId, String customerRef,
    BookingStatus status, int quantity,
    BigDecimal totalAmount, String currency,
    Instant expiresAt, Instant confirmedAt,
    List<BookingSeatResponse> seats,
    List<GeneralAdmissionResponse> gaLines,
    List<TicketResponse> tickets
) { }
```

EventSeatMapResponse

The response for GET /api/events/{id}/seat-map. Carries the hall layout AND live availability for every seat. This is the hottest read in the system — cached in Redis with TTL 2s and sync=true.

```
record EventSeatMapResponse(
    Long eventId, HallSummary hall,
    List<SectionAvailabilityResponse> sections,
    List<SeatAvailabilityResponse> seats,
    List<LayoutObjectResponse> layoutObjects
) { }
```

```
record SeatAvailabilityResponse(
    Long id, String label, Long sectionId,
    SeatAvailabilityStatus status,   // AVAILABLE | HELD | BOOKED
    BigDecimal price, String currency,
    Integer layoutX, Integer layoutY, /* ... */
) { }
```

GeneralAdmissionSelection

A line in a GA booking request. One section, one quantity.

```
public record GeneralAdmissionSelection(
    @NotNull Long sectionId,
    @Min(1) int quantity
) { }
```

ChangeSeatsRequest / ChangeBookingItemsRequest

Used by PUT /api/bookings/{id}/seats and /items. Lets a user swap seats on an existing HELD booking without re-starting the timer — the service preserves the original expiresAt.

Catalog DTOs

DTO	Purpose
CreateEventRequest	Admin creates a draft event. Takes name, hall, organizer, start/end times, tags.
EventResponse	Full event detail including pricing array and hall summary.
EventSummaryResponse	Compact row for the browse list.
CreateHallRequest	Admin creates a hall shell. Later edits layout via seat / section / layout-object endpoints.
HallResponse	Full hall detail: seats + sections + layout objects.
PricingItem / PricingResponse / SetEventPricingRequest	Per-section price overrides for an event.
SeatLayoutItem / SeatEditItem / UpdateHallSeatsRequest	Hall layout edit payloads.
SectionItem / SectionResponse	Section with polygon points.
LayoutObjectItem / LayoutObjectResponse	Stage/table/etc.
LayoutPresetRequest / Response / ItemSa	Save-and-reuse full hall layouts.
ShapeBox / PointItem	Geometry primitives shared across hall DTOs.
UpdateEventStatusRequest	DRAFT → ON_SALE transition.

Payment DTOs

DTO	Purpose
PaymentResponse	Status, amount, provider charge id, retry count.
PaymentResult (payment pkg)	Return value of the gateway abstraction — success + charge id OR failure + reason.
PaymentReviewItem	Admin view of a NEEDS_REVIEW payment that the reconciler gave up on.

Chapter 7 — Repositories

Spring Data JPA repositories are declared as interfaces. Most inherit standard CRUD methods for free (findById, save, delete, findAll). The notable methods are the custom queries that make the hot paths fast.

Catalog repositories

Repository	Key methods
EventRepository	findByStatus(status) for the browse list; findByIdForUpdate (PESSIMISTIC_WRITE) for th
HallRepository	findWithSeats(id), findWithEverything(id) — fetch joins to avoid N+1.
SeatRepository	findByHallIdOrderByRowIndexAscSeatNumberAsc — hot path, used by seat-map read.
SectionRepository	findByHallIdOrderByName.
EventPricingRepository	findByEventId, deleteByEventId.
OrganizerRepository, LayoutObjectRepository, LayoutPresetRepository	Standard CRUD + name lookups.

Reservation repositories

Repository	Key methods
BookingRepository	findByCustomerRef — 'my bookings'. sumConfirmedQuantity — capacity check. findExpir
BookingSeatRepository	findByEventIdAndStatusIn — seat-map availability query. findByBookingId — ticket issuar
PaymentRepository	findByBookingId, findByIdempotencyKey, findInDoubtOlderThan — reconciler input.
TicketRepository	findByBookingId, findByEventId — ticket retrieval.

The @Lock(LockModeType.PESSIMISTIC_WRITE) on findByIdForUpdate is the row-lock half of the concurrency story. Combined with the unique index on active_lock, it gives you two independent safeguards: the row lock serializes GA capacity counting (where no unique index can help, because GA has no identity), and the unique index makes seat double-booking impossible.

Chapter 8 — Services (expanded)

This chapter walks through the key methods of ReservationService with actual code and line-by-line commentary. It is the busiest file in the system — its public methods own the booking lifecycle from hold through payment, modification, cancellation, expiry, and reconciliation.

The service shape

```
@Service
public class ReservationService {
    private static final List<BookingSeatStatus> ACTIVE_SEAT_STATUSES =
            List.of(BookingSeatStatus.HELD, BookingSeatStatus.BOOKED);

    private static final int MAX_RECONCILE_ATTEMPTS = 6;
    private static final Duration RECONCILE_BASE_BACKOFF = Duration.ofSeconds(30);
    private static final Duration MAX_RECONCILE_BACKOFF = Duration.ofHours(1);
    private static final int RECONCILE_BATCH_SIZE = 200;
    private static final int SWEEP_BATCH_SIZE = 200;

    // 11 collaborators injected via constructor
    private final BookingRepository bookingRepository;
    private final BookingSeatRepository bookingSeatRepository;
    private final PaymentRepository paymentRepository;
    private final EventRepository eventRepository;
    private final SeatRepository seatRepository;
    private final TicketRepository ticketRepository;
    private final ReservationProperties properties;
    private final Clock clock;               // DatabaseAlignedClock injected
    private final CacheManager cacheManager;
    private final ObjectMapper objectMapper;
    private final UnpaidHoldThrottle unpaidHoldThrottle;
}
```

createBooking — the hot path

Isolation.READ_COMMITTED, event row lock + unique-index guard

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public BookingResponse createBooking(CreateBookingRequest request, CustomerId customer) {
    // 1. Lock the event row FOR UPDATE — serializes every GA capacity decision for this
event
    Event event = eventRepository.findByIdForUpdate(request.eventId())
        .orElseThrow(() -> ResourceNotFoundException.of("Event", request.eventId()));
    Instant now = clock.instant();

    // 2. Rate-limit check — 5 unpaid holds per 10 min per customer
    requireNoActiveHold(customer, now);

    // 3. The event must be PUBLISHED
    requireOpenForBooking(event);

    // 4. Validate and normalize the selection (dedupe seat ids, check quantities)
    Selection selection = normalizeSelection(request.seatIds(), request.generalAdmission());

    // 5. Release expired holds for THIS event so their seats become available for reuse.
    //    Flush: Hibernate orders INSERTs before UPDATEs in one flush; the release
    //    UPDATE (active_lock -> NULL) must reach the DB before the new hold's INSERT.
    releaseExpiredForEvent(event.getId(), now);
    bookingRepository.flush();

    // 6. Pre-check seats exist, belong to this hall, and belong to a SEATED section
```

```
    List<Seat> seats = resolveSeats(event, selection.seatIds());
    requireSeatsFree(event, selection.seatIds(), null);  // friendly 409 before the DB index
fires

    // 7. Check GA capacity under the event row lock
    Map<Long, Section> gaSections = resolveGaSections(event,
selection.gaQuantities().keySet());
    selection.gaQuantities().forEach((sectionId, quantity) ->
        requireGaCapacity(event, gaSections.get(sectionId), quantity, 0, now));

    // 8. Build the aggregate: booking + seat lines + GA lines + totals
    Booking booking = newBooking(event, customer, selection.total(), now);
    for (Seat seat : seats) {
        booking.addBookingSeat(newSeatLine(event, seat));
    }
    selection.gaQuantities().forEach((sectionId, quantity) ->
        booking.addGaLine(newGaLine(event, gaSections.get(sectionId), quantity)));
    recalculateTotals(booking);

    // 9. Save. If a seat was taken between pre-check and INSERT, the unique index throws
    try {
        bookingRepository.saveAndFlush(booking);
    } catch (DataIntegrityViolationException ex) {
        throw new ConflictException(
            "One or more selected seats were just taken. Please choose different seats.");
    }

    // 10. Register cache eviction — runs after this transaction commits
    evictAfterCommit(event.getId());
    return BookingResponse.from(booking);
}
```

Why both a pre-check (step 6) AND a unique-index catch (step 9)? The pre-check turns the common case — 'oh, that seat is already taken by someone else' — into a friendly error *before* the write. The unique index handles the race case — two inserts in the same millisecond — where pre-check can't help because both threads see 'free'.

cancelBooking

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public BookingResponse cancelBooking(Long bookingId, String customerRef) {
    Booking booking = getOwnedBookingForUpdate(bookingId, customerRef);
    if (booking.getStatus() != BookingStatus.PENDING_PAYMENT) {
        throw new BusinessRuleException(
            "Only a pending booking can be cancelled (status " + booking.getStatus() +
").");
    }
    booking.setStatus(BookingStatus.CANCELLED);
    releaseHeldSeats(booking);  // flips each HELD seat row to RELEASED → active_lock NULL
    evictAfterCommit(booking.getEvent().getId());
    return BookingResponse.from(booking);
}
```

A BOOKED cancellation is not handled here — that goes through a refund flow instead (not in this file), because it must coordinate with the payment provider.

changeItems — edit a pending hold

This is the most intricate method in the service. It must allow adding, removing, and re-quantifying items in a booking that is already PENDING_PAYMENT — without starting the timer over (that would let a user

keep inventory held forever by re-editing).

```
// 1. Only a pending booking can change
if (booking.getStatus() != BookingStatus.PENDING_PAYMENT) { throw ... }
if (booking.isExpired(now)) { throw ... }

// 2. The charged amount is fixed when payment begins — order must not change under it
paymentRepository.findByBookingId(booking.getId()).ifPresent(payment -> {
    if (payment.getStatus() != PaymentStatus.FAILED) {
        throw new ConflictException("The booking cannot change while a payment is processing
or unresolved.");
    }
});

// 3. Validate the new selection before touching anything
Selection selection = normalizeSelection(seatIds, generalAdmission);
releaseExpiredForEvent(event.getId(), now);
bookingRepository.flush();

// 4. Diff: which seats are NEW (need to be held), which are DROPPED (can be released)
Map<Long, BookingSeat> heldBySeat = new LinkedHashMap<>();
booking.heldSeats().forEach(bs -> heldBySeat.put(bs.getSeat().getId(), bs));
Set<Long> seatsToAdd = new LinkedHashSet<>(selection.seatIds());
seatsToAdd.removeAll(heldBySeat.keySet());
List<Seat> addedSeats = resolveSeats(event, seatsToAdd);
requireSeatsFree(event, seatsToAdd, booking.getId());  // allow seats this booking already
holds

// 5. Diff GA lines by section; only charge new capacity for increases
// ... quantity > held ? requireGaCapacity for delta : no check needed

// 6. Apply: release dropped seats, hold new ones, reshape GA
heldBySeat.forEach((seatId, line) -> {
    if (!selection.seatIds().contains(seatId)) line.setStatus(BookingSeatStatus.RELEASED);
});
for (Seat seat : addedSeats) booking.addBookingSeat(newSeatLine(event, seat));
booking.getGaLines().removeIf(line -> line.getSection() == null
    || !selection.gaQuantities().containsKey(line.getSection().getId()));
selection.gaQuantities().forEach(...);  // update existing or add new
recalculateTotals(booking);

// 7. Save with the same 409-on-race protection as createBooking
// 8. expiresAt is NOT modified — the original hold deadline stands
```

releaseExpired — the sweeper's work

Runs every 30s. Processes 200 bookings per transaction; the caller loops until a short batch returns.

```
@Transactional
public int releaseExpired() {
    Instant now = clock.instant();
    List<Booking> expired = bookingRepository.findExpiredHeldBatch(
        now, PageRequest.of(0, SWEEP_BATCH_SIZE));
    if (expired.isEmpty()) return 0;

    Set<Long> affectedEvents = new HashSet<>();
    for (Booking b : expired) {
        b.setStatus(BookingStatus.EXPIRED);
        // flip every HELD seat row to RELEASED, nulling its active_lock
        b.getSeats().stream()
            .filter(s -> s.getStatus() == BookingSeatStatus.HELD)
            .forEach(s -> s.setStatus(BookingSeatStatus.RELEASED));
        affectedEvents.add(b.getEvent().getId());
    }
```

```
    // evict the seat-map cache for every event that saw a release
    affectedEvents.forEach(this::evictAfterCommit);
    return expired.size();
}
```

Why bounded batches? After a flash sale, there might be 10,000 expired holds waiting. Pulling all of them into one transaction would hold row locks across thousands of updates — a lock storm. 200 per batch keeps each transaction short; the HoldSweeper wrapper loops until a batch comes back short, draining the backlog in bounded steps.

reconcile — the payment detective

Isolation.READ_COMMITTED, called by PaymentReconciliationJob

The reconciler's job is to answer a single question — 'is this payment resolved or not?' — and settle the booking state accordingly.

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public ReconcileOutcome reconcile(Long paymentId, Optional<PaymentResult> gatewayResult) {
    // Lock the event first (lock order: event → booking) to avoid deadlocks
    paymentRepository.findEventId(paymentId).ifPresent(eventId ->
            eventRepository.findByIdForUpdate(eventId));
    Instant now = clock.instant();
    Payment payment = paymentRepository.findById(paymentId).orElse(null);
    if (payment == null) return ReconcileOutcome.NONE;

    // Resume a refund already decided but not yet confirmed (worker died mid-flow)
    if (payment.getStatus() == PaymentStatus.REFUND_PENDING) {
        return ReconcileOutcome.refund(payment.getReference());
    }
    if (payment.getStatus() != PaymentStatus.INITIATED) {
        return ReconcileOutcome.NONE;
    }

    // Gateway says no successful charge → payment FAILED, booking stays PENDING_PAYMENT
    if (gatewayResult.isEmpty() || !gatewayResult.get().success()) {
        payment.setStatus(PaymentStatus.FAILED);
        payment.setFailureReason("No successful charge found during reconciliation.");
        return ReconcileOutcome.NONE;
    }

    // Gateway says charge SUCCEEDED → decide what to do with the booking
    String reference = gatewayResult.get().reference();
    Booking booking =
bookingRepository.findByIdForUpdate(payment.getBooking().getId()).orElseThrow(...);

    if (booking.getStatus() == BookingStatus.CONFIRMED) {
        // Already confirmed by another worker — just record the reference
        payment.setStatus(PaymentStatus.SUCCEEDED);
        payment.setReference(reference);
        return ReconcileOutcome.NONE;
    }
    if (booking.getStatus() == BookingStatus.PENDING_PAYMENT && !booking.isExpired(now)) {
        // Charge made it; hold still valid — confirm the booking now
        confirmPaidBooking(booking, payment, reference, now);
        return ReconcileOutcome.NONE;
    }

    // Charge succeeded but the hold is gone (expired or cancelled) — we owe a refund
    payment.setStatus(PaymentStatus.REFUND_PENDING);
    payment.setReference(reference);
    return ReconcileOutcome.refund(reference);
```

```
}
```

recordReconcileFailure — exponential backoff & dead-letter

```
payment.setReconcileAttempts(payment.getReconcileAttempts() + 1);
payment.setLastReconcileAt(clock.instant());
payment.setLastReconcileError(truncate(error));
if (payment.getReconcileAttempts() >= MAX_RECONCILE_ATTEMPTS) {
    payment.setStatus(PaymentStatus.NEEDS_REVIEW);
    log.error("Payment {} dead-lettered after {} failed reconciliations: {}",
            paymentId, payment.getReconcileAttempts(), payment.getLastReconcileError());
}
```

After 6 failed reconciliations (roughly one hour of exponential backoff), the payment is parked in NEEDS_REVIEW. An operator opens the admin's Payment Review page, investigates, and either refunds the customer directly or requeueForReconciliation after fixing the underlying issue.

requeueForReconciliation — operator action

```
@Transactional
public void requeueForReconciliation(Long paymentId) {
    Payment payment = paymentRepository.findById(paymentId).orElseThrow(...);
    if (payment.getStatus() != PaymentStatus.NEEDS_REVIEW) {
        throw new BusinessRuleException("Only a payment awaiting review can be requeued.");
    }
    payment.setStatus(PaymentStatus.INITIATED);
    payment.setReconcileAttempts(0);          // full budget again
    payment.setLastReconcileAt(null);
    payment.setLastReconcileError(null);
    log.info("Payment {} requeued for reconciliation by an operator.", paymentId);
}
```

getSeatMap — cached, sync, after-commit evicted

```
@Cacheable(cacheNames = CacheNames.EVENT_SEAT_MAP, key = "#eventId", sync = true)
@Transactional(readOnly = true)
public EventSeatMapResponse getSeatMap(Long eventId) {
    Event event = eventRepository.findById(eventId).orElseThrow(...);
    Hall hall = event.getHall();
    // Load everything in one shot: seats, sections, layout objects
    // Compute availability by overlaying active booking_seat rows for THIS event
    // Project to EventSeatMapResponse DTO
}
```

sync=true collapses a burst of concurrent misses for the same event into one loader call per JVM — the thundering-herd guard. TTL is 2 seconds (Redis level); eviction happens via evictAfterCommit(eventId) after every write path.

evictAfterCommit — the quiet safety trick

```
private void evictAfterCommit(Long eventId) {
    if (TransactionSynchronizationManager.isSynchronizationActive()) {
        TransactionSynchronizationManager.registerSynchronization(
            new TransactionSynchronization() {
                @Override public void afterCommit() {
                    evictAvailability(eventId);
                }
            });
    } else {
```

```
        evictAvailability(eventId);  // fallback — only hit outside @Transactional
    }
}
```

If the cache were evicted *before* the write transaction commits, a concurrent read could repopulate it with pre-commit state. The afterCommit hook guarantees the eviction only fires once the write is visible to other transactions — zero stale-read window under normal conditions.

Chapter 9 — Business flows (expanded)

This chapter walks through nine business flows with sequence diagrams and plain-English narration. Each flow is a single logical business operation, even when it spans multiple HTTP calls.

Flow 1 — Hold a seat (happy path)

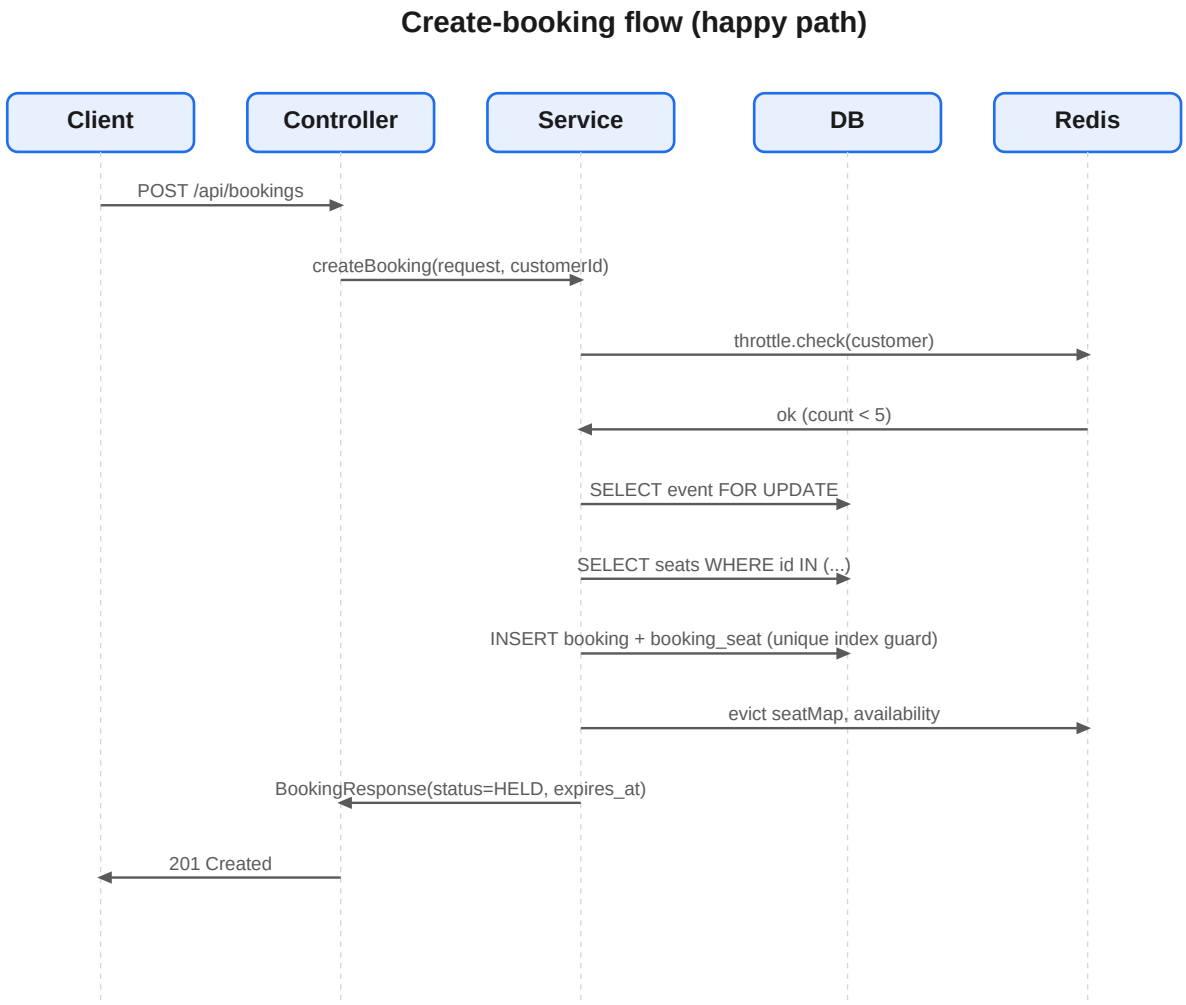


Figure 9.1 — Successful hold.

- Client POSTs /api/bookings with { eventId, seatIds, gaLines }.
- Controller resolves the customer via @CurrentCustomer.
- Service checks the rate limit (5 unpaid holds / 10 min).
- Service acquires an event row lock (SELECT ... FOR UPDATE).
- Service releases any expired holds for this event, then flushes — so their seats are freed before the new insert tries to grab them.
- Service inserts the booking + booking_seat rows. MySQL computes active_lock; if it collides with any existing HELD/BOOKED row, the unique index throws ER_DUP_ENTRY, the service catches it, returns 409.

- Cache eviction registered as an afterCommit hook.
- Service returns BookingResponse with status PENDING_PAYMENT and expiresAt = now + 10 min.

Flow 2 — Pay for a hold

Two phases: synchronous charge attempt, then (if network timed out) async reconciliation.

- Client POSTs /api/bookings/{id}/payment with a payment-method token.
- Service verifies ownership (booking.customer_ref == current customer).
- Service creates a Payment row with status=INITIATED, idempotency_key='booking-{id}-{attempt}'.
- Service calls PaymentGateway.charge(context).
- **Success**: booking → CONFIRMED, seats → BOOKED (active_lock stays — seat permanently locked), tickets issued, cache evicted, throttle wiped. Response: 200 with tickets.
- **Failure**: payment → FAILED, booking stays PENDING_PAYMENT. User can start another attempt, which bumps the idempotency key's attempt number so the provider doesn't replay the decline.
- **Timeout**: payment stays INITIATED. Reconciler picks it up ~1 min later.

Flow 3 — Abandoned hold (sweeper)

HoldSweeper runs every 30s:

```
@Scheduled(fixedDelayString = "${app.reservation.sweep-interval}")
public void sweep() {
    if (!lease.acquire("hold-sweeper", Duration.ofMinutes(5))) return;
    int released;
    do {
        released = reservationService.releaseExpired();
    } while (released == SWEEP_BATCH_SIZE);  // loop until batch is short
}
```

SchedulerLease uses Redis SET NX EX to ensure only one app instance sweeps at a time — three replicas don't each release the same expired holds.

Flow 4 — Payment in doubt (reconciler)

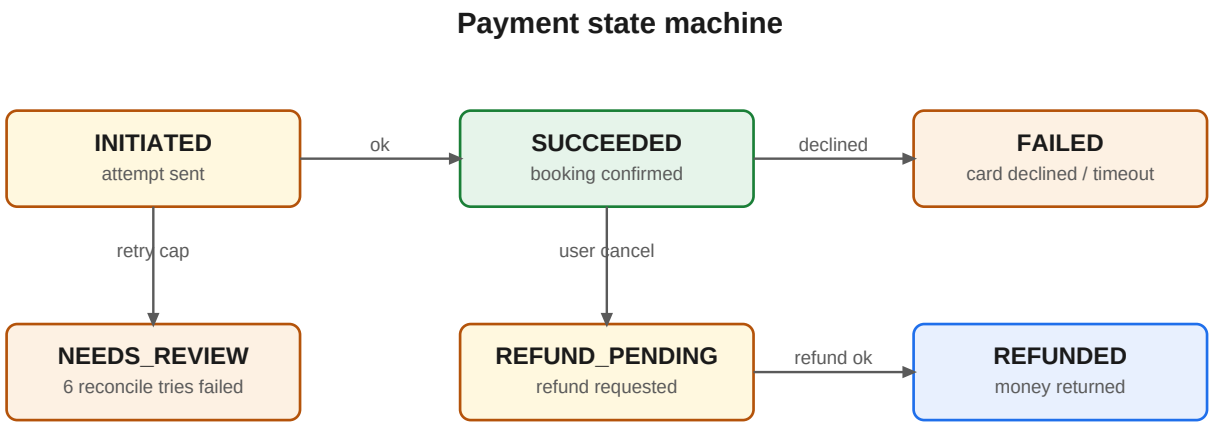


Figure 9.2 — Payment state machine.

A payment left in INITIATED for longer than reconcile-after (1 min) is 'in doubt'. The reconciler asks the provider 'is this charge resolved?' via the idempotency key, then:

- **Gateway says SUCCEEDED, booking still PENDING_PAYMENT and not expired** → confirm the booking.
- **Gateway says SUCCEEDED, booking CONFIRMED already** → just record the reference.
- **Gateway says SUCCEEDED, booking EXPIRED/CANCELLED** → owe a refund; status → REFUND_PENDING; emit ReconcileOutcome.refund(reference). An outer worker issues the refund and calls markRefunded.
- **Gateway says FAILED or 'charge not found'** → payment FAILED; booking stays PENDING_PAYMENT.
- **Gateway unreachable** → recordReconcileFailure increments retry_count, sets exponential backoff (30s → 1h cap).
- **6 failed reconciles** → status NEEDS_REVIEW; surfaces in admin's Payment Review page.

Flow 5 — Browse and view seat map

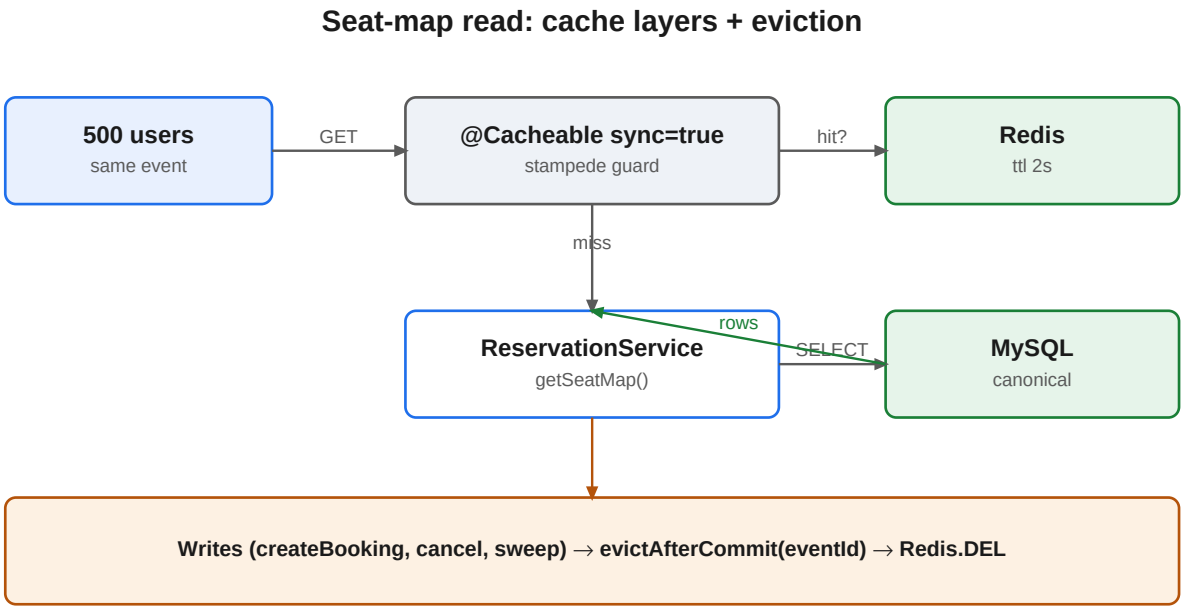


Figure 9.3 — Seat-map read.

Flow 6 — Refund after booking cancellation

When a CONFIRMED booking is cancelled (customer changed their mind, admin flagged fraud), the money must come back. This flow spans two transactions and may run asynchronously.

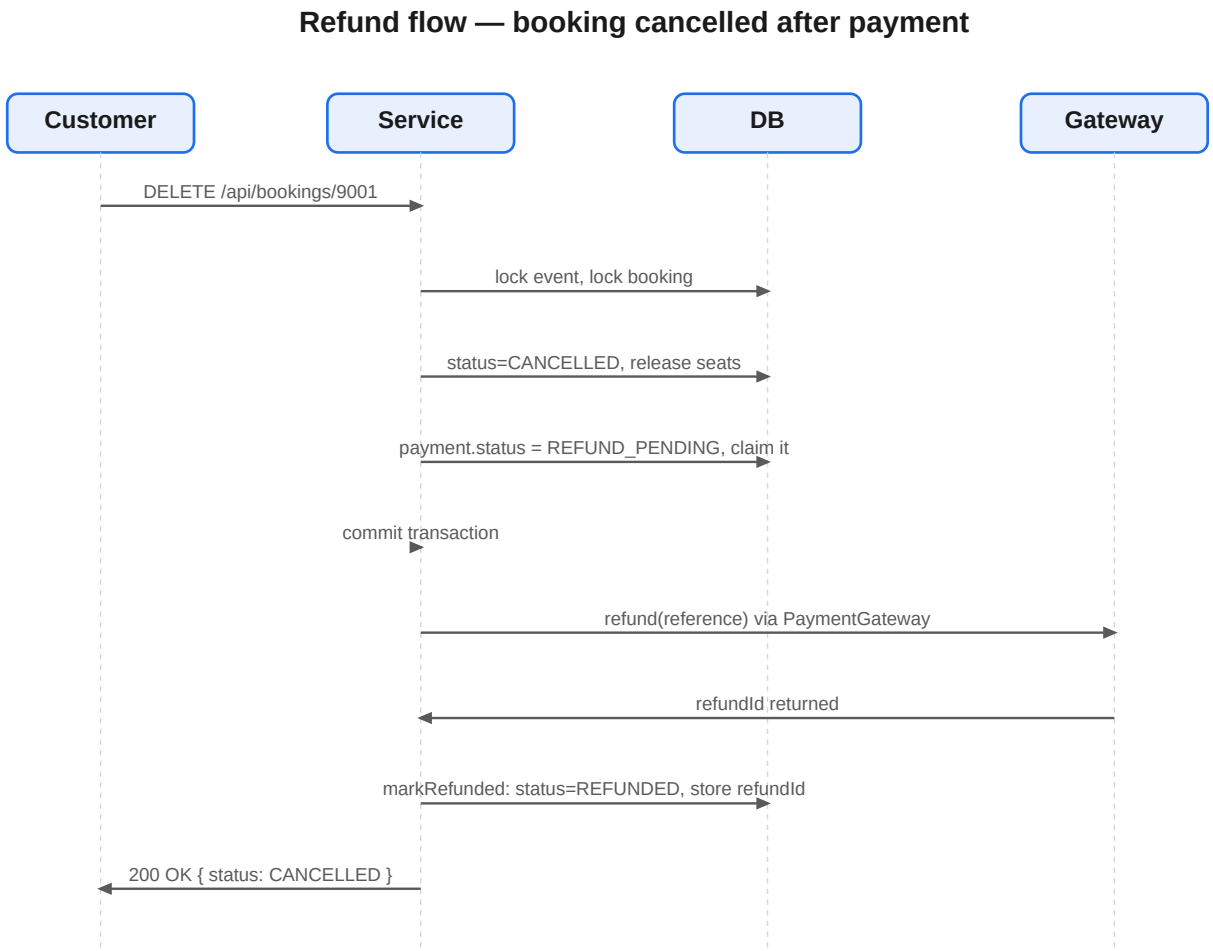


Figure 9.4 — Refund flow.

- Service marks booking CANCELLED, releases seats (the row lock prevents concurrent re-booking).
- Payment moves from SUCCEEDED → REFUND_PENDING. The REFUND_PENDING claim is stored BEFORE the actual refund call, so no other replica/tick can decide the same refund twice.
- Service calls PaymentGateway.refund(reference) outside the transaction (so a hang doesn't hold DB locks).
- On success → markRefunded: status → REFUNDED, reference stored.
- On failure → payment stays REFUND_PENDING. The reconciler sees it next tick and retries from the REFUND_PENDING resume path.

Flow 7 — Modify a hold without restarting the timer

User has A1 held, decides they also want A2 and swap the Lawn quantity from 2 to 4. The booking's expiresAt is preserved — a user can't keep inventory held forever by re-editing.

- PUT /api/bookings/{id}/items with the new { seatIds, gaLines }.
- Service locks booking + event. Refuses if any payment is in flight (INITIATED or SUCCEEDED).
- Diffs old vs new: identifies seats to add, seats to drop, GA deltas.
- Validates: new seats free, GA capacity for increases only.

- Applies: drops release to RELEASED, new ones hold with fresh active_lock, GA lines reshape in place.
- Totals recalculated; expiresAt untouched.
- Cache evicted after commit.

Flow 8 — Admin publishes an event

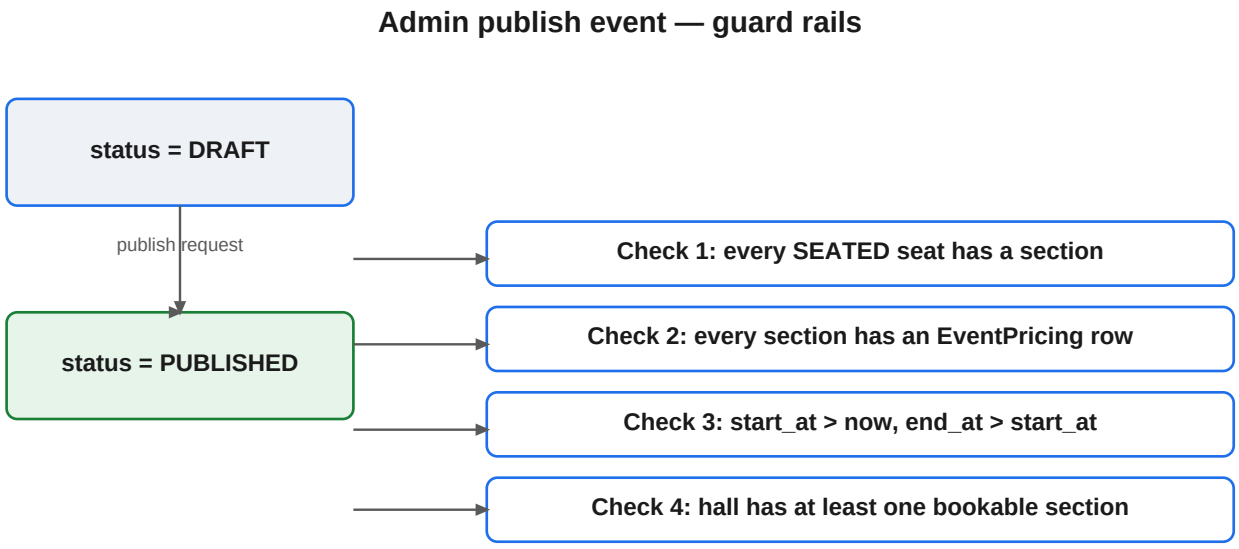


Figure 9.5 — Publish guard rails.

Publishing moves an event from DRAFT to PUBLISHED. The service enforces four invariants before accepting the transition:

Check	Why
Every SEATED seat has a section	A seat without a section has no price and no geometry overlay — it would appear 'nowhere' to
Every section has an EventPricing row	A section without a price would crash the seat-map response. Snapshotting from an absent ro
start_at > now, end_at > start_at	A past event or an end-before-start event is a data bug surfaced at publish time.
Hall has at least one bookable section	An event with zero bookable sections is a draft, not a sellable thing.

Flow 9 — Apply a layout preset

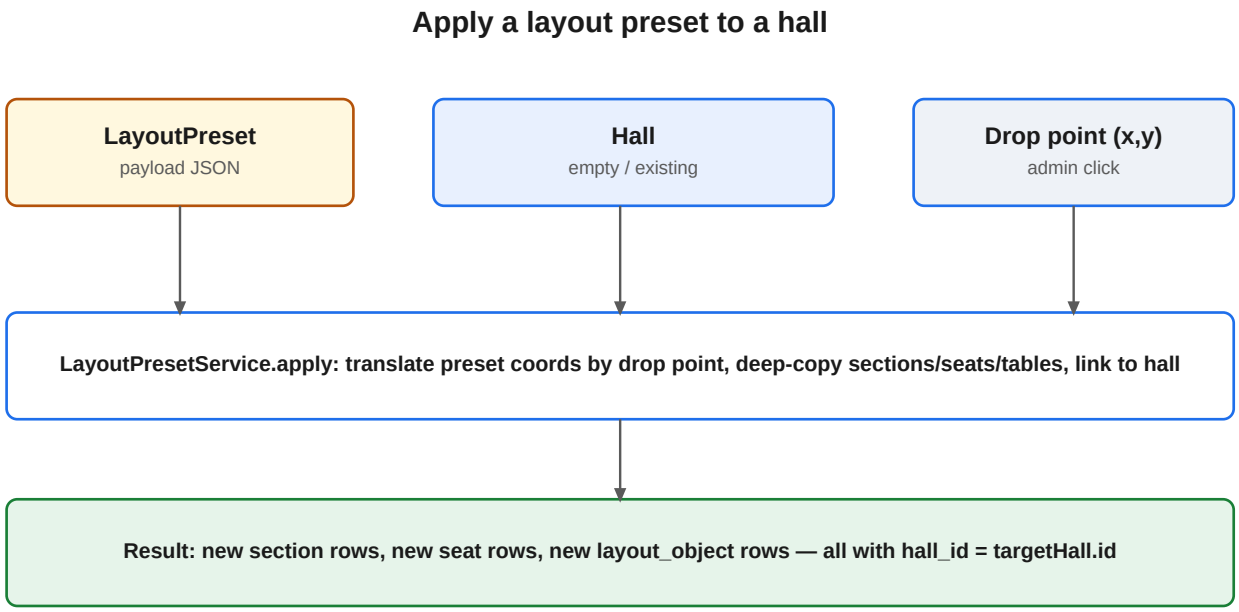


Figure 9.6 — Layout preset application.

An admin designs a hall once, saves it as a preset (sections + seats + layout objects + their relative coordinates), and stamps it onto new halls. The preset is hall-agnostic — its coordinates are relative to its own bounding box, so stamping is a simple translate by the drop point. Sections, seats, and layout objects are deep-copied so later edits to one hall don't ripple to others.

The booking state machine (recap)

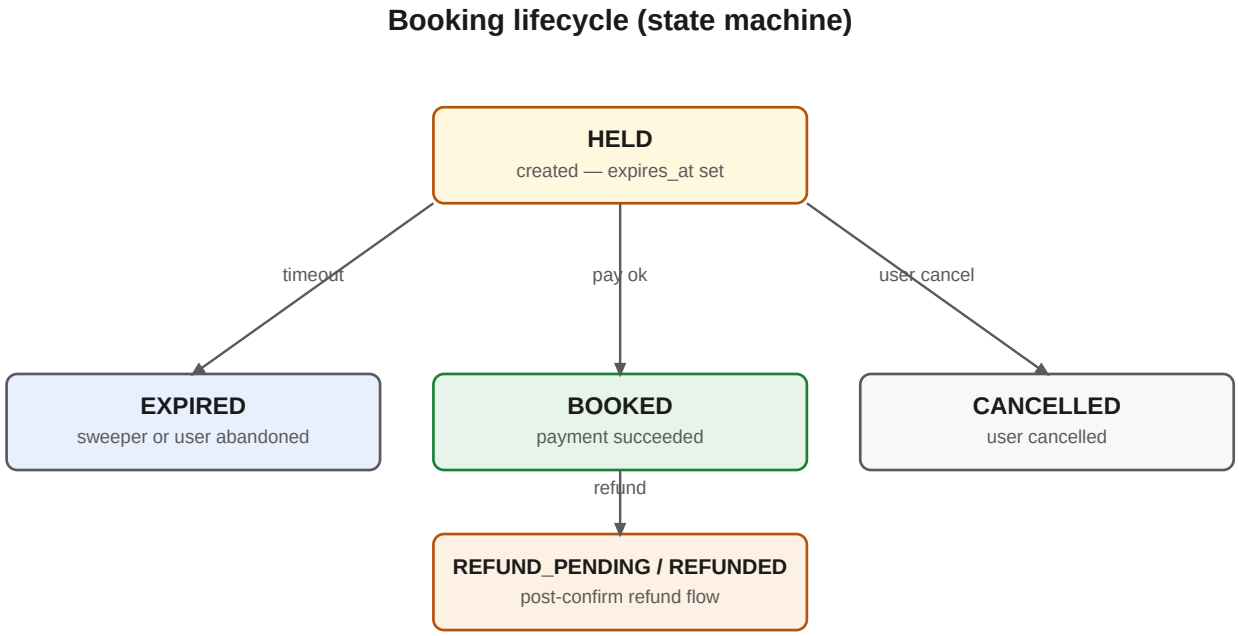


Figure 9.7 — Full booking lifecycle with transitions.

Chapter 10 — Controllers (REST API)

Controllers are the HTTP boundary. They are thin: validate the input, resolve the current customer, call a service method, return the response. No business logic lives here. Every exception is caught by GlobalExceptionHandler and turned into an ApiError JSON body with a clean HTTP status.

Mobile controllers (/api/**)

BookingController — /api/bookings

Method	Path	Purpose
POST	/api/bookings	Create a hold. Throttle applies.
GET	/api/bookings	List my bookings.
GET	/api/bookings/{id}	Get one booking (owner only).
PUT	/api/bookings/{id}/items	Change the seat list on a HELD booking (preserves expiresAt).
PUT	/api/bookings/{id}/seats	Simpler seat swap.
POST	/api/bookings/{id}/payment	Pay for a HELD booking.
DELETE	/api/bookings/{id}	Cancel (HELD or BOOKED).

EventBrowseController — /api/events

Method	Path	Purpose
GET	/api/events	List ON_SALE events (browse).
GET	/api/events/{id}	Event detail + pricing array.
GET	/api/events/{id}/seat-map	Full seat map with live availability. Cached 2s.
GET	/api/events/{id}/availability	Lightweight section-level counts.

Admin controllers (/admin/**)

Controller	Endpoints	Purpose
EventAdminController	POST /admin/events, PUT /admin/events/{id}, PUT /admin/events/{id}/status, PUT /admin/events/{id}/pricing	CRUD + lifecycle + pricing
HallController	POST /admin/halls, PUT /admin/halls/{id}, PUT /admin/halls/{id}/layout	Build halls, edit basic info, replace layout
LayoutPresetController	POST /admin/presets, POST /admin/presets/{id}/apply/{hallId}	Save and apply hall layout templates.
OrganizerController	CRUD /admin/organizers	Manage event owners.
PaymentReviewController	GET /admin/payments/review, POST .../resolve	Human-in-the-loop for NEEDS_REVIEW payments.

Upload endpoint

UploadController at POST /admin/uploads/images accepts multipart image files (JPEG, PNG, WebP, GIF), caps size at 10 MB, stores in uploads/, returns a URL. UploadWebMvcConfig serves uploads back at /uploads/**. In production, the uploads/ folder must be a persistent volume.

Error handling

Every endpoint that throws routes to GlobalExceptionHandler, which returns:

Exception	HTTP	Body example
ResourceNotFoundException	404	{ "code": "NOT_FOUND", "message": "Booking 42 not found" }
ConflictException (seat taken)	409	{ "code": "SEAT_CONFLICT", "message": ... }
BusinessRuleException	422	{ "code": "RULE_VIOLATED", ... }
TooManyRequestsException	429	{ "code": "RATE_LIMITED", "retryAfter": 120 }
UnauthorizedException	401	{ "code": "UNAUTHORIZED" }
MethodArgumentNotValidException	400	{ "code": "VALIDATION", "fieldErrors": [...] }
Any other RuntimeException	500	{ "code": "INTERNAL_ERROR" } — details logged, not leaked.

Chapter 11 — Cross-cutting concerns

Authentication (current shape)

The system uses a lightweight 'customer identity' pattern instead of Spring Security:

- @CurrentCustomer — parameter annotation placed on controller methods. Signals 'inject the current customer here'.
- CustomerId — immutable record carrying a UUID. ref() returns the string form stored on booking.customer_ref.
- CurrentCustomerArgumentResolver — Spring HandlerMethodArgumentResolver. Reads an Authorization: Bearer <token> header, verifies it with TokenAuthenticator, returns a CustomerId.
- TokenAuthenticator — validates HMAC-signed JWT-ish tokens.
- DevTokenController — POST /api/dev/token mints a valid token for any identity. Dev-only; must be off in production.
- WebMvcConfig — registers the resolver with Spring MVC.

Important: four of these six classes are currently **deleted** in the working tree (CurrentCustomerArgumentResolver, TokenAuthenticator, WebMvcConfig, DevTokenController). The annotation and the record remain, but with no resolver registered the DispatcherServlet cannot fulfill @CurrentCustomer parameters and would 500 on every booking endpoint. This is the top-priority gap.

Rate limiting

RateLimiter is a general-purpose fixed-window counter backed by Redis. UnpaidHoldThrottle is a specialization aimed at booking abuse.

RateLimiter

```
public boolean check(String key, int limit, Duration window) {
    String bucketKey = "rate:" + key + ":" + currentBucket(window);
    long count = redisTemplate.opsForValue().increment(bucketKey);
    if (count == 1L) {
        redisTemplate.expire(bucketKey, window);
    }
    // if (count > limit) { throw new TooManyRequestsException(...); }  // commented out
    return count <= limit;
}
```

The throw at lines 63–67 is **commented out**. The limiter counts attempts but does not reject. Flipping it back on is a one-line change; keep it in mind as a live gap.

UnpaidHoldThrottle

Builds on RateLimiter to track *unpaid* holds per customer. Parameters from application.yml: unpaid-hold-limit: 5, unpaid-hold-window: PT10M. On successful payment, onPaymentConfirmed(customerRef) wipes the tally so a real repeat buyer is never blocked.

Caching

Two Redis caches, both for read-only snapshots of event availability:

Cache name	Keyed by	TTL	Loaded by
eventSeatMap	eventId	2s	ReservationService.getSeatMap
eventAvailability	eventId	2s	ReservationService.getAvailability

Both use @Cacheable(sync=true) to prevent cache stampede and are evicted after commit on every write path. See Chapter 9 Flow 5 for the full behavior.

Scheduled jobs

Job	Interval	Lease	What it does
HoldSweeper	30s	SchedulerLease	Flips HELD → EXPIRED for timed-out bookings, 200 at a time.
PaymentReconciliationJob	60s	SchedulerLease	Resolves INITIATED payments older than reconcile-after; dead-l

SchedulerLease uses Redis SET NX EX to ensure only one instance runs the job in a multi-instance deployment. Without it, three replicas would each sweep the same rows every 30s.

CORS

WebConfig maps /api/**, allows any origin pattern, methods GET/POST/PUT/PATCH/DELETE/OPTIONS, and does NOT allow credentials. Suitable for a mobile app that uses a bearer token. Not suitable if you add a browser-based admin that uses session cookies on the same endpoints.

Exception handling

GlobalExceptionHandler (@ControllerAdvice) maps every custom exception to a JSON ApiError. Unknown runtime exceptions become 500 with the message hidden (logged server-side).

Chapter 12 — Configuration reference

Every tunable, from application.yml, in one place:

Spring

Property	Value	Why
spring.servlet.multipart.max-file-size	12 MB	Outer bound for one upload request.
spring.datasource.hikari.maximum-pool-size	20	Concurrency ceiling; must stay well below MySQL max_conn
spring.datasource.hikari.minimum-idle	5	Warm connections ready to serve.
spring.datasource.hikari.connection-timeout	3000 ms	Fail fast under load — a 3s wait is already lost.
spring.datasource.hikari.leak-detection-threshold	20000 ms	Logs if a connection is held >20s.
spring.jpa.hibernate.ddl-auto	validate	Hibernate must NEVER rewrite the schema — Flyway is the s
spring.jpa.open-in-view	false	No lazy loading inside controllers; forces explicit fetch.
spring.data.redis.timeout	500 ms	Cache ops degrade to a DB read rather than hang.

Reservation

Property	Value	Why
app.reservation.hold-duration	PT10M	How long a seat stays HELD before auto-release.
app.reservation.max-seats-per-booking	10	Defense in depth behind CreateBookingRequest.MAX_SEAT
app.reservation.sweep-interval	PT30S	How often HoldSweeper runs.
app.reservation.cache-ttl	PT2S	Seat-map Redis TTL.
app.reservation.unpaid-hold-limit	5	How many unpaid holds in the window before 429.
app.reservation.unpaid-hold-window	PT10M	Window for unpaid-hold count.
app.reservation.reconcile-after	PT1M	How long INITIATED payments wait before reconciliation.

Security & uploads

Property	Value	Why
app.security.dev-tokens-enabled	true (dev) / false (prod)	Must be false in production.
app.uploads.directory	./uploads	Where event posters are written. Mount a volume i
app.seed-demo-data	true (dev)	DemoDataSeeder loads a sample catalog on startu

Chapter 13 — Known gaps and risks

Honest list of what is incomplete, fragile, or trade-off territory, in priority order.

Priority 1 — Blocking

#	Gap	Impact	Fix
1	@CurrentCustomer resolver and token-issuing endpoint deleted at Phase 0	Every booking endpoint 500s at runtime	Restore CurrentCustomerArgumentResolver + WebMvcC
2	RateLimiter.check() does not throw — brute-force attack is not blocked	Hold-spam attack is not blocked	Uncomment the throw at RateLimiter.java:63-67

Priority 2 — Fragility

#	Gap	Impact	Fix
3	No CUSTOMER entity — only customer_ref string	Cannot store email, preferences, or loyalty data	Add Customer entity + FK; migrate customer_ref in a batc
4	seat.section_name denormalized, data-only	Admin renames section; past bookings still show old name	Either drop the column and JOIN, or add a trigger to keep in sync
5	ticket.event_id and ticket.seat_id are plain BIGINTs, not FKs	Can create tickets referencing deleted entities	Add FK constraints
6	EventPricing has no UNIQUE(event_id, section_id)	Admin can insert two conflicting prices for the same section	Add the composite unique constraint
7	Section.points stored as TEXT JSON, not a spatial type	Cannot use MySQL's point-in-polygon in SQL; must compute in Java	Migrate to MySQL POLYGON if point-in-polygon becomes

Priority 3 — Scale risks (hypothetical)

#	Gap	When it bites
8	Event row lock as flash-sale bottleneck	10,000 concurrent GA requests serialize on the event row
9	No partition or archival strategy for booking / booking_ticket	Past five years of bookings sit in one hot table
10	Cache stampede across multiple app instances	Spring's @Cacheable sync=true is per-JVM; 3 replicas = 3 concurrent DB qu
11	CORS allows any origin	Fine for mobile; risky if browser admin shares endpoints

Priority 4 — Code quality / future refactors

#	Gap	Why it's still fine
12	Enum columns tight (length 16, 20)	Adding a longer enum value needs a Flyway migration first.
13	No automated test for 'every write path calls evictAvailabilityCache'	A new controller method could ship without cache eviction; 2s TTL hides it in dev.
14	FakePaymentGateway returns success randomly	Fine for dev; real integration (Stripe / Checkout.com) is still to do.

None of these are showstoppers for a v1. The two 'blocking' items in Priority 1 are mechanical: restoring the deleted resolver is a straight-forward Spring MVC config; uncommenting the throw is literally a one-line change. Everything else below Priority 2 is work for scale that most real systems happily defer.

Appendix A — Glossary

Simple definitions of every term used in this system, in alphabetical order.

Term	Meaning
active_lock	A generated column on booking_seat. Equals '{event_id}-{seat_id}' while the row is HELD or BOOKE
afterCommit hook	A Spring hook that runs code only after the surrounding transaction commits successfully. Used to ev
BigDecimal(12,2)	Database type for all monetary values. Twelve total digits, two after the decimal. No floats — floats lo
Booking	An order. Groups seats and/or GA quantity lines for one customer on one event.
BookingSeat	A single held or booked seat within a booking. Carries the active_lock generated column.
BookingGaLine	A general-admission line within a booking. Quantity instead of individual seat rows.
Cache stampede	Thundering herd. When a popular cache key expires, many concurrent requests all miss simultaneou
CustomerId	A record carrying a UUID. ref() returns the string form stored on booking.customer_ref.
DDL	Data Definition Language — the SQL that creates tables and indexes. Managed by Flyway here.
Dead-letter row	A row in payment_dead_letter for a payment the reconciler gave up on after 6 attempts. Surfaces in t
Event	A single performance. Has one hall and one organizer. Prices are overlays on the hall's sections.
EventPricing	Per-section price for one event. The hall knows the venue; the event decides how much today's perfo
Expired hold	A booking still in HELD status whose expires_at is in the past. The sweeper flips it to EXPIRED and r
Flyway	Database migration tool. Each V*.sql file is a versioned migration applied once per database and nev
GA	General Admission. A booking mode where tickets are quantity-based, not seat-based. Standing are
Generated column	A MySQL column computed from an expression on other columns. STORED means the value is writ
Hold	A temporary reservation. Lasts 10 minutes. Blocks the seat so other users can't buy it while this user
Idempotency key	A unique string sent with a payment attempt so the gateway treats retries as the same operation, not
Instant	Java's UTC timestamp. All server times use it. Converted to local time only for display.
Lock order	The fixed sequence in which locks are acquired to prevent deadlocks. Here: event row first, then boo
Optimistic lock	The version column. Hibernate adds WHERE version=? to every update; if the row has been change
Pessimistic lock	SELECT … FOR UPDATE. Holds a row-level write lock for the duration of the transaction. Used on t
Reconciliation	The background job that resolves payments stuck in INITIATED status. Asks the gateway 'did this ac
Row lock	See 'Pessimistic lock'.
Section	A first-class named region of a hall. VIP, Balcony, Lawn. Has booking mode (SEATED or GA), curren
Seat	A single individually-sellable spot. Belongs to a hall directly. Section is an optional overlay set by geo
Section_name snapshot	A denormalized copy of the section's name stored on booking_seat and booking_ga_line at hold time
Snapshot (in a booking line)	Copying price, currency, and section name from the catalog onto the booking row at hold time. The b
Sweeper	The scheduled job (HoldSweeper) that releases expired holds. Runs every 30s, processes 200 rows
sync=true	On @Cacheable, enables per-key serialization of concurrent misses. One loader runs; others wait fo
Throttle	Specifically UnpaidHoldThrottle — rate-limits hold attempts per customer. 5 per 10 minutes, cleared
Ticket	An issued ticket, one per seat or GA unit, created when a booking is confirmed. Carries a signed QR
TTL	Time To Live — how long a cache entry lives before it expires. Here: 2 seconds for the seat map.
Unique index (nullable)	MySQL allows unlimited NULLs in a UNIQUE column. We use this so RELEASED rows don't collide
version column	Optimistic lock counter on every entity. Hibernate increments on update.

Appendix B — API examples

Real request/response examples for the most-used mobile endpoints.

Browse events

GET /api/events?category=concert&featured;=true

```
// 200 OK
[
  {
    "id": 42,
    "name": "Fairuz Night at the Roman Theatre",
    "category": "concert",
    "imageUrl": "/uploads/evt-42-cover.jpg",
    "tagline": "An evening with the legend",
    "startAt": "2026-12-20T20:00:00Z",
    "venue": "Roman Theatre",
    "minPrice": "25.00",
    "currency": "JOD",
    "status": "ON_SALE"
  }
]
```

Get seat map

GET /api/events/42/seat-map

```
// 200 OK — cached in Redis for 2s
{
  "eventId": 42,
  "hall": { "id": 7, "name": "Roman Theatre", "numRows": 20, "numColumns": 30 },
  "sections": [
    { "id": 11, "name": "VIP",     "bookingMode": "SEATED", "price": "80.00", "currency":
"JOD",
      "color": "#2E75DC", "points":
"[{x:120,y:40},{x:600,y:40},{x:600,y:180},{x:120,y:180}]" },
    { "id": 12, "name": "Lawn",    "bookingMode": "GENERAL_ADMISSION", "capacity": 500,
      "available": 312, "price": "25.00", "currency": "JOD" }
  ],
  "seats": [
    { "id": 2001, "label": "A1", "sectionId": 11, "status": "AVAILABLE",
      "price": "80.00", "currency": "JOD", "layoutX": 130, "layoutY": 50 },
    { "id": 2002, "label": "A2", "sectionId": 11, "status": "HELD",  ... },
    { "id": 2003, "label": "A3", "sectionId": 11, "status": "BOOKED", ... }
  ],
  "layoutObjects": [
    { "type": "STAGE", "x": 200, "y": 10, "width": 400, "height": 30 }
  ]
}
```

Create a mixed booking (seats + GA)

POST /api/bookings

```
// Request
{
  "eventId": 42,
  "seatIds": [2001, 2002],
  "gaLines": [
```

```
    { "sectionId": 12, "quantity": 4 }
  ]
}

// 201 Created
{
  "id": 9001,
  "eventId": 42,
  "status": "HELD",
  "quantity": 6,
  "totalAmount": "260.00",
  "currency": "JOD",
  "expiresAt": "2026-12-04T19:30:00Z",   // now + 10 min
  "seats": [
    { "seatId": 2001, "label": "A1", "sectionName": "VIP", "price": "80.00", "status":
"HELD" },
    { "seatId": 2002, "label": "A2", "sectionName": "VIP", "price": "80.00", "status":
"HELD" }
  ],
  "gaLines": [
    { "sectionId": 12, "sectionName": "Lawn", "quantity": 4, "unitPrice": "25.00" }
  ],
  "tickets": []
}
```

Possible failure responses:

```
// 409 Conflict — seat A1 was just taken by someone else
{ "code": "SEAT_CONFLICT", "message": "Seat A1 is no longer available" }

// 422 Unprocessable Entity — GA section is sold out
{ "code": "RULE_VIOLATED", "message": "Lawn has only 2 units available" }

// 429 Too Many Requests — rate limit
{ "code": "RATE_LIMITED", "message": "Too many unpaid holds", "retryAfterSeconds": 300 }
```

Pay for a booking

POST /api/bookings/9001/payment

```
// Request
{
  "paymentMethodToken": "tok_abc123"
}

// 200 OK — synchronous success
{
  "paymentId": 5001,
  "status": "SUCCEEDED",
  "providerChargeId": "ch_xyz789",
  "bookingStatus": "BOOKED",
  "tickets": [
    { "ticketId": 7001, "label": "A1", "qrPayload": "..." },
    { "ticketId": 7002, "label": "A2", "qrPayload": "..." },
    { "ticketId": 7003, "label": "Lawn", "qrPayload": "..." }, ...
  ]
}

// 202 Accepted — in doubt (will be reconciled)
{ "paymentId": 5001, "status": "INITIATED", "bookingStatus": "HELD" }

// 402 Payment Required — card declined
{ "code": "PAYMENT_FAILED", "message": "Card declined", "canRetry": true }
```

Cancel a booking

DELETE /api/bookings/9001

```
// 200 OK — HELD booking cancelled, seats released immediately
{ "status": "CANCELLED" }

// 200 OK — BOOKED booking: refund flow kicked off
{ "status": "CANCELLED", "paymentStatus": "REFUND_PENDING" }
```

Appendix C — Concurrency deep-dive

This appendix answers: 'if two users click the same seat at the same millisecond, exactly what happens?'

The race

Two threads enter createBooking concurrently with the same seat ID.

```
// Thread A                                  // Thread B
BEGIN TRANSACTION                            BEGIN TRANSACTION
throttle.check(userA)      OK                throttle.check(userB)      OK
SELECT event FOR UPDATE    (acquires)        SELECT event FOR UPDATE    (BLOCKED)
SELECT seat 2001           (status free)
INSERT booking             (id=100)
INSERT booking_seat        (seat=2001, booking=100, status=HELD)
     -- MySQL computes active_lock='42-2001' and writes the row
COMMIT
                                             // (lock released, B wakes up)
                                             SELECT event FOR UPDATE    (acquires)
                                             SELECT seat 2001  (status still shows FREE in B's
snapshot)
                                             INSERT booking             (id=101)
                                             INSERT booking_seat        (seat=2001, booking=101,
status=HELD)
                                                  -- active_lock='42-2001' COLLIDES with A's row
                                                  -- InnoDB rejects: ER_DUP_ENTRY
                                             ROLLBACK
                                             // service catches, translates to 409 Conflict
```

Two independent safeguards

The design uses BOTH a row lock AND a unique index — on purpose:

Mechanism	Protects what	Why we need it
Event row lock (FOR UPDATE)	GA capacity counting	GA tickets have no identity — you can't put a unique index on 'the 3rd g
Unique index on active_lock	Seat double-booking	Seats have identity. The database can physically reject a duplicate inser

What this design does NOT protect against

- **Clock skew between app instances**: expires_at is computed in app clock, read in DB clock. The DatabaseAlignedClock bean adjusts for drift.
- **A rogue admin SQL update**: if someone manually sets status='HELD' on an EXPIRED row, they bypass every guard. There is no audit table yet.
- **Payment provider replay**: if the gateway accepts the same idempotency key twice as separate charges (violating their own contract), the reconciler can't detect it.

The **reason** the active_lock column is GENERATED and STORED (not virtual) is that only STORED columns can be the target of a UNIQUE index in MySQL InnoDB. Virtual columns are recomputed every read and can't be indexed. This is a MySQL-specific detail that would read differently in Postgres (which supports partial unique indexes, so you'd write UNIQUE (event_id, seat_id) WHERE status IN ('HELD','BOOKED') directly).

Appendix D — FAQ

Why Instant instead of LocalDateTime for event start times?

Instant is UTC-only and unambiguous. LocalDateTime has no timezone and silently assumes server-local time — a bug waiting to happen when servers move regions.

Why BigDecimal for money instead of long cents?

Both work. BigDecimal is explicit about precision and scale, serializes cleanly to JSON as a string ('80.00'), and avoids silent rounding. Long cents require every display path to divide by 100 and remember the currency's decimal places (JPY has zero, BHD has three).

Why cascade ALL + orphanRemoval everywhere?

Booking is an aggregate root. Deleting a booking must delete its seats, GA lines, tickets, and payment attempts — they have no meaning without it. The cascade enforces that at the ORM level; the FKs enforce it at the DB level.

Why not use UUIDs as primary keys?

BIGINT is 8 bytes; UUID is 16 bytes. On the hot table (booking_seat), that doubles every FK, every index, and every WHERE lookup. Auto-increment BIGINTs are the right default; add a public_id UUID column only if you need to expose IDs to untrusted parties without leaking order counts.

Why MySQL and not PostgreSQL?

Historical choice. Both would work. MySQL gets us generated columns and STORED unique indexes; Postgres would get us partial unique indexes (cleaner for active_lock) and JSONB (cleaner for section.points). Migrating is a quarter of work, not a weekend.

Why Redis for the rate limiter instead of in-memory?

Multi-instance. If you run three app replicas and the counter is in-memory, an attacker rotates traffic across replicas and each sees only 1/3 of their holds. Redis gives you a single truth.

Why not use Spring Security?

Overhead and complexity for what is effectively 'read a JWT, inject an identity.' The current pattern — a custom HandlerMethodArgumentResolver — does the one thing Spring Security would do and nothing else. The trade-off is you don't get method-level @PreAuthorize, CSRF for free, or the OAuth filter chain. Fine for mobile-first; add Spring Security the day you add a browser admin.

Why is the reconciler not using a message queue?

A queue would be cleaner, but the current scale doesn't need one. A @Scheduled job that polls a small table every minute is 100 lines of code and no new infrastructure. Introduce Kafka / RabbitMQ when you have a second service that cares about the same events.

What's the deployment topology?

Single Spring Boot JAR + MySQL + Redis + optional object storage for uploads. The admin app (../admin/) is a separate Spring Boot deploy. Everything can run as docker-compose today or be split into Kubernetes workloads later.

How do I add a new booking status?

1) Add the enum value in BookingStatus. 2) Write a Flyway migration to widen the status column if needed (it's VARCHAR(20)). 3) Update every switch/if that handles BookingStatus — the compiler will mostly catch these. 4) Make sure the sweeper, reconciler, and seat-map filter know whether this status counts as 'active.'

Why isn't there an email service?

Deliberately deferred. The Ticket entity carries the QR payload; the delivery channel is a separate concern that can be added later. The system doesn't currently store customer emails either (see 'no CUSTOMER entity' gap in Chapter 13).